

MATH 8090: ARMA Models-Inference, Diagnostics, and Model Selection

Whitney Huang, Clemson University

9/21-9/25/2026

Contents

MLE	1
Lake Huron Example	1
CSS, MLE, and CSS-MLE	2
Model selection	3
Model Diagnostics	4
Monte Carlo Simulation	5

MLE

Lake Huron Example

```
# Load the Lake Huron annual water-level time series
data("LakeHuron")

# Extract the observation years
yr <- time(LakeHuron)

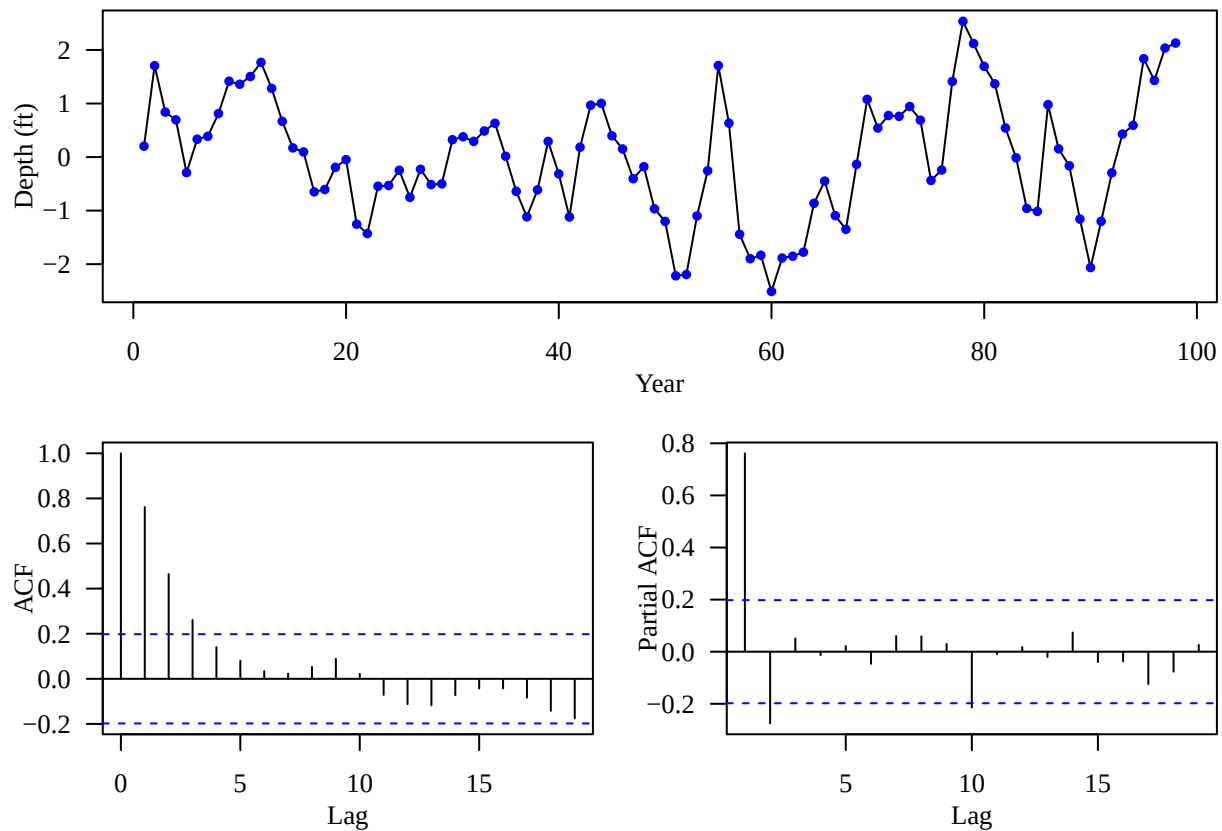
# Fit a linear trend model
lm <- lm(LakeHuron ~ yr)

# Set graphical parameters and arrange three diagnostic plots
par(las = 1, mgp = c(2, 1, 0), mar = c(3.6, 3.6, 0.8, 0.6), family = "serif")
layout(matrix(c(1, 1, 2, 3), 2, 2, byrow = TRUE))

# Plot residuals over time to assess remaining temporal structure
plot(lm$residuals, ylab = "Depth (ft)", xlab = "Year", type = "l")
points(lm$residuals, cex = 0.8, col = "blue", pch = 16)

# Plot the sample ACF of the residuals
acf(lm$residuals)

# Plot the sample PACF of the residuals
pacf(lm$residuals)
```



CSS, MLE, and CSS-MLE

```
# Fit an AR(2) model using conditional sum of squares (CSS)
(CSS <- arima(lm$residuals, order = c(2, 0, 0), include.mean = FALSE,
              method = "CSS"))

##
## Call:
## arima(x = lm$residuals, order = c(2, 0, 0), include.mean = FALSE, method = "CSS")
##
## Coefficients:
##          ar1      ar2
##       1.0020  -0.2834
## s.e.  0.0947   0.0964
##
## sigma^2 estimated as 0.4436:  part log likelihood = -99.23

# Fit an AR(2) model using exact maximum likelihood (ML)
(MLE <- arima(lm$residuals, order = c(2, 0, 0), include.mean = FALSE,
              method = "ML"))

##
## Call:
## arima(x = lm$residuals, order = c(2, 0, 0), include.mean = FALSE, method = "ML")
```

```
##
## Coefficients:
##          ar1      ar2
##      1.0050  -0.2925
## s.e.  0.0976   0.1002
##
## sigma^2 estimated as 0.4572:  log likelihood = -101.26,  aic = 208.51

# Fit an AR(2) model using the default CSS-ML approach:
# CSS provides initial parameter estimates, followed by ML estimation
(CSS_ML <- arima(lm$residuals, order = c(2, 0, 0), include.mean = FALSE))

##
## Call:
## arima(x = lm$residuals, order = c(2, 0, 0), include.mean = FALSE)
##
## Coefficients:
##          ar1      ar2
##      1.0050  -0.2925
## s.e.  0.0976   0.1002
##
## sigma^2 estimated as 0.4572:  log likelihood = -101.26,  aic = 208.51
```

The three methods produce similar AR(2) parameter estimates, but use different estimation procedures. CSS minimizes the conditional sum of squared errors, whereas ML maximizes the exact Gaussian likelihood. The default CSS-ML method uses CSS to obtain starting values and then performs ML estimation; therefore, its final estimates should generally agree with those from `method = "ML"`.

Model selection

```
# Specify candidate ARMA(p,q) models for the errors
orders <- list(
  c(1, 0, 0), # ARMA(1,0)
  c(1, 0, 1), # ARMA(1,1)
  c(2, 0, 0), # ARMA(2,0)
  c(2, 0, 1) # ARMA(2,1)
)

# Create model labels
models <- c("ARMA(1,0)", "ARMA(1,1)", "ARMA(2,0)", "ARMA(2,1)")

# Fit each ARMA model with a linear time trend as an external regressor
fit <- lapply(orders, function(z) arima(LakeHuron, order = z, xreg = yr))
names(fit) <- models

# Compare candidate models using AIC
lapply(fit, AIC)

## $'ARMA(1,0)'
## [1] 218.4501
##
```

```
## $'ARMA(1,1)'
## [1] 212.3954
##
## $'ARMA(2,0)'
## [1] 212.3965
##
## $'ARMA(2,1)'
## [1] 214.0638
```

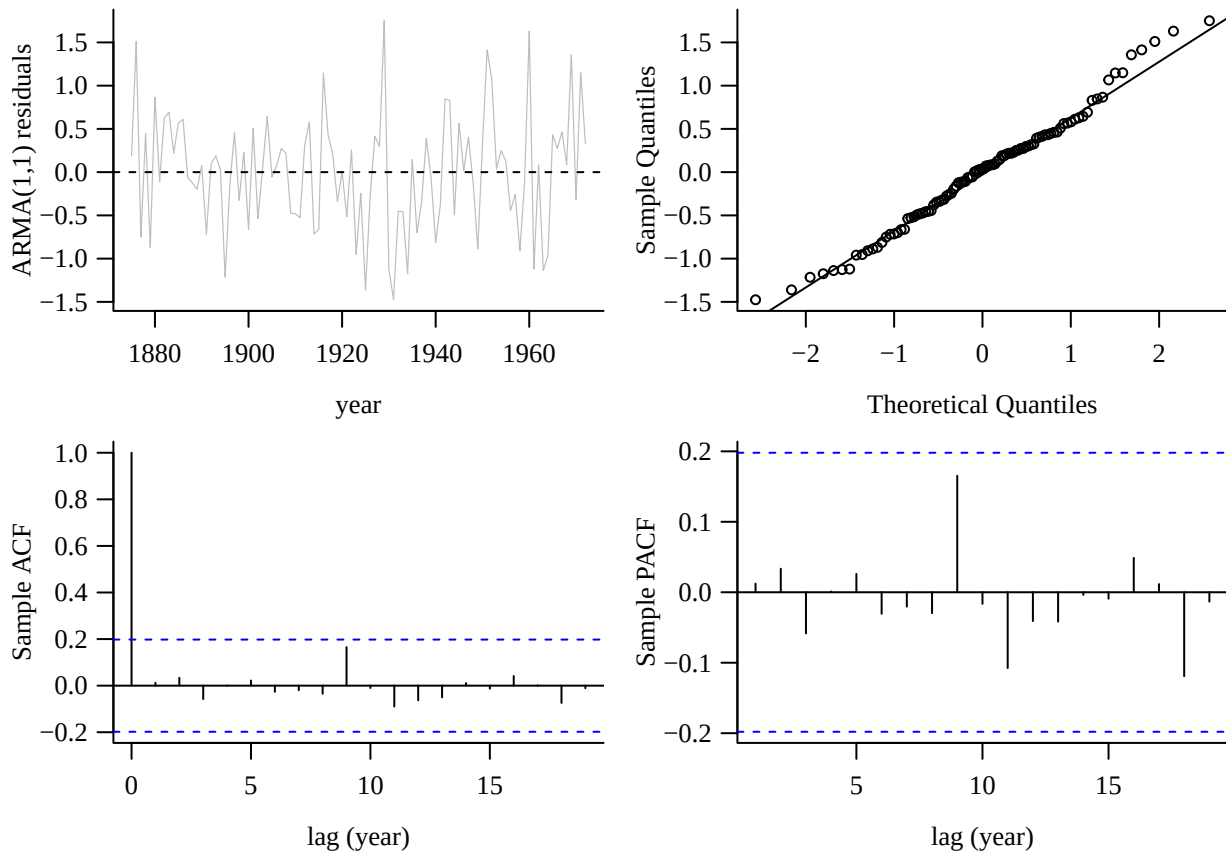
```
# Compare candidate models using the small-sample corrected AIC (AICc)
library(MuMin)
lapply(fit, AICc)
```

```
## $'ARMA(1,0)'
## [1] 218.8803
##
## $'ARMA(1,1)'
## [1] 213.0476
##
## $'ARMA(2,0)'
## [1] 213.0487
##
## $'ARMA(2,1)'
## [1] 214.9868
```

We fit several candidate ARMA models while jointly estimating the linear trend and the ARMA error structure. AIC and AICc balance goodness of fit against model complexity, with smaller values indicating a preferred model. AICc applies an additional finite-sample penalty and is particularly useful when the sample size is relatively small.

Model Diagnostics

```
## extract the residuals
resids <- resid(fit[[2]])
## time series plot of the residuals
par(bty = "L", mar = c(3.6, 3.6, 0.5, 0.6), las = 1, mgp = c(2.4, 0.8, 0),
    mfrow = c(2, 2), family = "serif")
plot(resids, xlab = "year",
     ylab = "ARMA(1,1) residuals", lwd = 0.6, col = "gray")
abline(h = 0, lty = 2)
## Normal Q-Q plot for the residuals
qqnorm(resids, main = "", cex = 0.75); qqline(resids)
## Sample ACF and PACF of the residuals
acf(resids, ylab = "Sample ACF", xlab = "lag (year)", main = "")
pacf(resids, ylab = "Sample PACF", xlab = "lag (year)")
```



```
## Carry out the Ljung-Box test
Box.test(resids, lag = 10, type = "Ljung-Box", fitdf = 2)
```

```
##
## Box-Ljung test
##
## data:  resids
## X-squared = 3.7882, df = 8, p-value = 0.8757
```

After fitting the ARMA(1,1) model, we examine the residuals to assess model adequacy. A well-fitting model should leave residuals that behave approximately like white noise: no systematic temporal pattern, little remaining autocorrelation, and—under the Gaussian ARMA assumption—an approximately normal distribution. The Ljung–Box test provides a formal test for remaining serial dependence; a large p -value indicates no evidence against the white-noise assumption.

Monte Carlo Simulation

We use a Monte Carlo simulation to compare the finite-sample performance of Yule–Walker (YW) and maximum likelihood estimation (MLE) for an AR(2) model. We generate $N = 1000$ independent time series, each of length $n = 100$, from

$$Y_t = 0.6Y_{t-1} + 0.35Y_{t-2} + W_t,$$

and estimate ϕ_1 and ϕ_2 from each simulated series.

```
## Number of Monte Carlo replications and length of each time series
N <- 1000
n <- 100

## True AR(2) parameters
phi <- c(0.6, 0.35)

## Generate N independent AR(2) time series, each of length n
## Each column of sim1 contains one simulated time series
sim1 <- replicate(N, arima.sim(n = n, list(ar = phi)))
```

For each simulated time series, we fit an AR(2) model using both Yule-Walker estimation and maximum likelihood estimation. This gives N sets of parameter estimates from each method.

```
## Fit an AR(2) model to each simulated series using Yule-Walker
yw <- apply(sim1, 2, ar, aic = FALSE, order.max = 2, method = "yule-walker")

## Fit an AR(2) model to each simulated series using maximum likelihood
mle <- apply(sim1, 2, ar, aic = FALSE, order.max = 2, method = "mle")
```

Next, we extract the estimated AR coefficients. Each row corresponds to one Monte Carlo replication, while the two columns contain the estimates of ϕ_1 and ϕ_2 .

```
## Extract the Yule-Walker estimates of phi_1 and phi_2
yw_phi <- t(array(unlist(lapply(yw, function(x) x$ar))), dim = c(2, N))

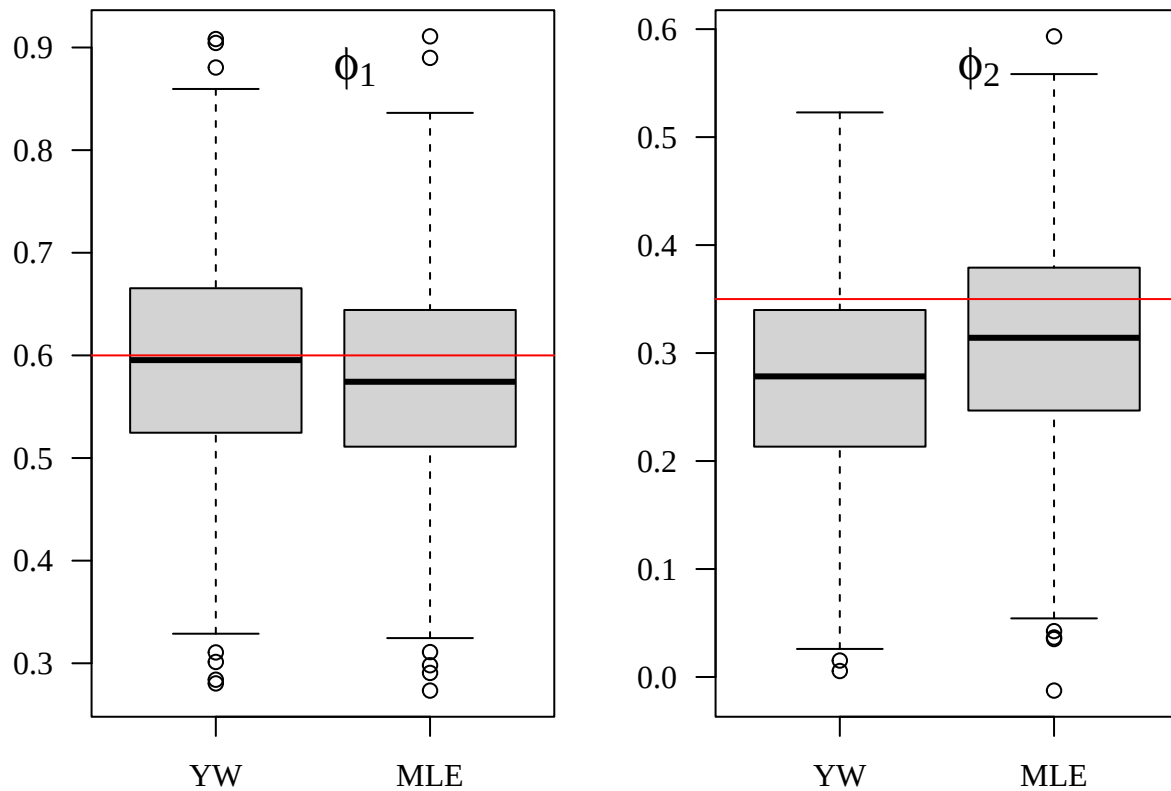
## Extract the MLE estimates of phi_1 and phi_2
mle_phi <- t(array(unlist(lapply(mle, function(x) x$ar))), dim = c(2, N))
```

We compare the sampling distributions of the two estimators using boxplots. The horizontal red lines indicate the true parameter values, $\phi_1 = 0.6$ and $\phi_2 = 0.35$. An estimator whose distribution is tightly concentrated around the true value has desirable finite-sample performance.

```
## Compare the Monte Carlo sampling distributions of YW and MLE
par(mar = c(3.6, 3.6, 0.5, 0.6), las = 1,
    mgp = c(2.2, 1, 0), mfrow = c(1, 2), family = "serif")

## Sampling distributions of phi_1 estimates
boxplot(yw_phi[, 1], mle_phi[, 1], xaxt = "n")
abline(h = 0.6, col = "red")
axis(1, at = 1:2, labels = c("YW", "MLE"))
legend("top", legend = expression(phi[1]), bty = "n", cex = 1.5)

## Sampling distributions of phi_2 estimates
boxplot(yw_phi[, 2], mle_phi[, 2], xaxt = "n")
abline(h = 0.35, col = "red")
axis(1, at = 1:2, labels = c("YW", "MLE"))
legend("top", legend = expression(phi[2]), bty = "n", cex = 1.5)
```



We can quantify the comparison using the Monte Carlo mean and standard deviation of the estimates. The mean indicates potential finite-sample bias, while the standard deviation measures sampling variability.

```
## Monte Carlo means and standard deviations for Yule-Walker
apply(yw_phi, 2, mean)
```

```
## [1] 0.5959340 0.2772561
```

```
apply(yw_phi, 2, sd)
```

```
## [1] 0.10447045 0.09272058
```

```
## Monte Carlo means and standard deviations for MLE
apply(mle_phi, 2, mean)
```

```
## [1] 0.5781796 0.3124750
```

```
apply(mle_phi, 2, sd)
```

```
## [1] 0.1006118 0.0962791
```

We can also compare the estimators using root mean squared error (RMSE). RMSE accounts for both bias and sampling variability; smaller RMSE indicates more accurate estimation.

```
## RMSE for estimating phi_1 using Yule-Walker
sqrt(mean((yw_phi[, 1] - 0.6)^2))
```

```
## [1] 0.1044973
```

```
## RMSE for estimating phi_1 using MLE
sqrt(mean((mle_phi[, 1] - 0.6)^2))
```

```
## [1] 0.1029016
```

Finally, we examine whether the estimated AR(2) models satisfy the causality (stationarity) condition. For an estimated AR(2),

$$\phi(z) = 1 - \hat{\phi}_1 z - \hat{\phi}_2 z^2,$$

the fitted model is causal if both roots of $\phi(z) = 0$ lie outside the unit circle, i.e., have modulus greater than 1.

```
## Compute the moduli of the roots of the estimated AR polynomial
## for each Yule-Walker fit
roots <- t(apply(yw_phi, 1, function(x)
  Mod(polyroot(c(1, -x[1], -x[2])))))

## Flag noncausal fitted models:
## 0 = both roots are outside the unit circle (causal)
## 1 = at least one root is on or inside the unit circle (noncausal)
check <- apply(roots, 1, function(x)
  ifelse(x[1] > 1 && x[2] > 1, 0, 1))

## Number and proportion of noncausal estimates
sum(check)
```

```
## [1] 0
```

```
mean(check)
```

```
## [1] 0
```

This simulation illustrates the finite-sample behavior of AR parameter estimators. Repeated simulation lets us empirically examine bias, variability, and RMSE of Yule-Walker and MLE rather than relying only on their large-sample properties. It also illustrates an important practical issue: after estimation, we should check whether the fitted AR coefficients correspond to a causal process.